

Progetto di programmazione avanzata e parallela

Parte 2 di 2 - Linguaggio Python

Lo scopo del progetto è quello di produrre un piccolo renderer 2D in stile retro. Il sistema deve utilizzare una palette indicizzata di 16 colori, una tile map per il fondale e un insieme di sprite sovrapposti al fondale stesso.

I tile sono immagini di dimensione 32×32 pixel, gli sprite sono immagini di dimensione 64×64 pixel, e tutti i colori usati fanno riferimento alla stessa palette di 16 colori, ovvero ogni immagine inizialmente è composta solo da 4bit per pixel che indicizzano il colore da usare in una palette di 16 colori (un array con 16 colori, uno per posizione)

La configurazione della scena deve essere salvata in un file JSON, mentre il risultato finale deve essere esportato come immagine PNG, per questo può essere usata la libreria Pillow.



La parte rimanente di questo testo illustrerà l'architettura del renderer, il formato dei file di input e le componenti da implementare.

Architettura del renderer

Il renderer è composto dalle seguenti componenti:

1. Una palette di 16 colori. Ogni colore è rappresentato come una terna RGB di interi tra 0 e 255. Tutti i pixel dei tile e degli sprite contengono soltanto un indice nella palette, quindi un valore tra 0 e 15.
2. Un tile sheet di dimensione 256×256 . Esso contiene 64 tile organizzati in una griglia 8×8 , ciascuno di dimensione 32×32 . Il tile con identificatore 0 occupa la regione in alto a sinistra, il tile con identificatore 1 la regione immediatamente successiva sulla stessa riga, e così via.
3. Uno sprite sheet di dimensione 256×256 . Esso contiene 16 sprite organizzati in una griglia 4×4 , ciascuno di dimensione 64×64 . Anche in questo caso gli identificatori sono assegnati in ordine riga per riga, a partire dall'angolo in alto a sinistra.
4. Un frame buffer di dimensione 640×480 . Il fondale occupa tutto lo schermo tramite una tile map di 20 colonne e 15 righe, dato che $640/32 = 20$ e $480/32 = 15$.
5. Un indice di trasparenza per gli sprite. Quando un pixel di uno sprite contiene tale indice, il relativo pixel non deve essere copiato nel frame buffer e il contenuto già presente deve rimanere invariato.
6. Un insieme di trasformazioni per gli sprite. Ogni sprite può essere ribaltato orizzontalmente, ribaltato verticalmente e ruotato di 0, 90, 180 o 270 gradi. Le trasformazioni vanno applicate allo sprite prima del disegno sul frame buffer.



L'ordine di composizione è il seguente: prima si disegna l'intero fondale usando la tile map, poi si disegnano gli sprite nell'ordine in cui compaiono nel JSON della scena. Gli sprite possono trovarsi anche parzialmente fuori dallo schermo; in tal caso devono essere disegnati soltanto i pixel che ricadono dentro il frame buffer.

Formato dei file di input

Oltre a produrre il renderer, si deve essere anche in grado di leggere i dati della scena e degli asset da file esterni.

La palette è salvata in un file JSON contenente esattamente 16 colori rappresentato come un array JSON puro. Ad esempio:

```
[
  [0, 0, 0],
  [255, 0, 0],
  [0, 255, 0],
  [0, 0, 255],
  [255, 255, 255],
  [255, 255, 0],
  [255, 128, 0],
  [128, 128, 128],
  [0, 255, 255],
  [255, 0, 255],
  [128, 0, 0],
  [0, 128, 0],
  [0, 0, 128],
  [128, 128, 0],
  [128, 0, 128],
  [0, 128, 128]
]
```



Il tile sheet e lo sprite sheet sono salvati come matrici indicizzate 256×256 in formato binario packed. Questo significa che il file sarà composto da $(256 \times 256)/2$ bytes, dato che ogni byte contiene due pixel: i 4 byte più alti corrispondono al primo pixel e i 4 più bassi al secondo pixel. Quindi, per rimarcare, dato che l'intera matrice contiene 65536 pixel, ciascun file binario ha dimensione esattamente pari a 32768 byte.



La scena è salvata in un file JSON con tre componenti principali:

1. **transparent_index**, ovvero l'indice della palette che deve essere considerato trasparente per gli sprite (ma non per il fondale!).
2. **tile_map**, ovvero una matrice di 15 righe e 20 colonne contenente gli identificatori dei tile da disegnare sul fondale.
3. **sprites**, ovvero una lista di sprite. Ogni sprite possiede almeno i campi





id, x, y, flip_h, flip_v e rotation. Questi campi indicano rispettivamente l'id della sprite da utilizzare (la sua posizione nello sprite sheet), la posizione in cui metterla e se fare flip rispettivamente orizzontale o verticale e se, successivamente, ruotare (solo di 0, 90, 180 e 270 gradi, nessun altro valore è consentito).

Un esempio di file di scena completo è il seguente:

```
{
  "transparent_index": 0,
  "tile_map": [
    [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
    [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
    [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
    [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
    [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
    [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
    [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
    [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
    [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
    [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
    [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
    [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
    [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
    [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
    [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
    [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
    [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
  ],
  "sprites": [
    {"id": 0, "x": 32, "y": 32,
     "flip_h": false, "flip_v": false, "rotation": 0},
    {"id": 0, "x": 128, "y": 32,
     "flip_h": true, "flip_v": false, "rotation": 0},
    {"id": 2, "x": 224, "y": 32,
     "flip_h": false, "flip_v": true, "rotation": 0},
    {"id": 1, "x": 320, "y": 32,
     "flip_h": false, "flip_v": false, "rotation": 90}
  ]
}
```

Si consiglia di sfruttare i file di esempio allegati al progetto per testare e comprendere meglio il funzionamento del renderer e del formato di input.

Per leggere i file json si utilizzi la libreria `json` che è parte della libreria standard python, che converte facilmente da file a dizionari/liste Python.

Interfaccia a riga di comando

Il programma deve essere eseguibile da riga di comando con la seguente forma:

```
python main.py <palette.json> <scene.json> <tiles.bin> <sprites.bin> <output.png>
```

L'ordine degli argomenti è il seguente: 1. percorso del file JSON della palette; 2. percorso del file JSON della scena; 3. percorso del file binario del tile sheet; 4. percorso del file binario dello sprite sheet; 5. percorso del file PNG di output.

Non sono previste altre modalità di utilizzare il render che sostituiscano l'interfaccia da linea di comando (quindi niente inserimento interattivo dei valori).

Classi da implementare

1. Una classe **Palette** che permetta, dato un file JSON, di caricare e validare una palette di 16 colori e di risolvere un indice nel corrispondente valore RGB. 
2. Una classe **VirtualVRAM** che permetta di caricare il tile sheet e lo sprite sheet dai rispettivi file binari, decodificando le sequenze di 4 bit (solitamente chiamate “nibble”) in una matrice di indici di palette.
3. Una classe **SceneParser** che permetta di caricare un file JSON di scena e restituire una descrizione completa del fondale e degli sprite.
4. Una classe **Blitter** che permetta di estrarre tile e sprite dai rispettivi sheet, applicare flip, rotazioni e trasparenza, e copiare i risultati nel frame buffer. 
5. Una classe **RenderingPipeline** che permetta di eseguire la composizione completa della scena, convertire il frame buffer indicizzato in RGB e salvare il risultato finale come immagine PNG. 

Potete inserire in un file README come far funzionare il renderer da riga di comando e come lanciare eventuali test o esempi forniti.

Requisiti

- In caso di errore il codice deve generare delle eccezioni coerenti con il tipo di condizione che ha provocato l'errore. È possibile definire delle eccezioni apposite o usare quelle predefinite (si sconsiglia sollevare eccezioni troppo generiche, come **Exception**).
- Ogni funzione rilevante (i.e., non di 2-3 righe) deve essere adeguatamente commentata spiegando che compito svolge, che input richiede e che output genera.
- Pillow può essere usata esclusivamente per convertire il risultato finale in PNG.
- Usate numpy per lavorare su array, in particolare sfruttate il tipo `np.uint8`.
- Il programma deve poter essere eseguito da riga di comando esclusivamente nella forma `python main.py <palette.json> <scene.json> <tiles.bin> <sprites.bin> <output.png>`.

- Ogni file Python deve contenere all’inizio come commento il proprio nome, cognome e numero di matricola.

Consegna

Le date di consegna sono **sempre** esattamente da 10 a 7 giorni prima dell’appello *orale*. Quindi se un orale è, per esempio, il giorno 18, sarà possibile consegnare i progetti del giorno 8 (10 giorni prima) al giorno 11 (7 giorni prima). Non inviate al di fuori delle date di consegna: le consegne fuori da questo intervallo di date **non** verranno considerate.

Istruzioni per la consegna:

- Preparare il progetto e salvarlo come un file `.zip` o `.tar.gz` contenente tutti i file *sorgenti* necessari. **Non saranno accettati altri formati di archivio**. Possono essere allegati piccoli file di esempio strettamente necessari a mostrare il funzionamento del renderer (ad esempio JSON e asset binari delle texture). Non devono invece essere allegati output PNG generati automaticamente. Può essere allegato (ed è anzi consigliato) un file `README.txt` o `README.md` con istruzioni e note.
- Il file di archivio *deve* avere nome `Cognome Nome Matricola.zip` o `Cognome Nome Matricola.tar.gz`
- Il file di archivio *deve* avere due directory all’interno chiamate `C` e `Python` (rispettate le maiuscole e minuscole) contenenti rispettivamente il progetto `C` e il progetto `Python`.
- Inviare una mail a `lmanzoni@units.it` con il seguente oggetto: `[Consegna Progetto Programmazione Avanzata e Parallela] Cognome Nome Matricola` usando la vostra email istituzionale. Dovete chiaramente allegare il progetto, per il testo della mail non vi sono vincoli.
- Il mancato rispetto di questi vincoli (e.g., allegare file come archivi RAR, non rispettare il nome delle directory e dei file) può comportare l’automatico annullamento del progetto.



Note importanti

- Questo progetto è relativo solamente alla parte in Python del corso. Per sostenere la parte di progetto del corso è necessario consegnare sia la parte in C (parte 1) che la parte in Python (parte 2).
- Sebbene appaia ovvio, quando viene chiesto di inserire Nome, Cognome e Matricola, dovete inserire il vostro nome, cognome e numero di matricola e non, letteralmente, il testo “Nome”, “Cognome” e “Matricola”.
- Il progetto rimane valido anche per gli appelli successivi, non è necessario inviarlo nuovamente.
- Ogni consegna successiva annulla quella precedente, si sconsiglia però di inviare decine di versioni durante il periodo di consegna.
- Devono essere sempre consegnate entrambe le parti, **NON** solo una delle due.

- Verificate se il progetto è stato aggiornato con chiarimenti, questo avviene se diverse persone fanno osservazioni che si richiede siano da chiarire per tutti.